UNIDAD DIDÁCTICA 02

Desarrollo de módulos en Odoo_

Python, ORM, XML, seguridad y extensión de aplicaciones estándar

Módulos	ORM	Vistas XML	Herencia

Odoo 18 Community · Entorno local Docker de la UD1

Guía de contenidos

En esta unidad dejamos de usar Odoo únicamente como usuarios y empezamos a tratarlo como una plataforma de desarrollo. Todo el hilo conductor se construye sobre PokeStore, una tienda ficticia de figuras coleccionables Pokémon.

Bloque	Qué aprenderás
01	Qué es un módulo y cómo encaja en Odoo
02	Estructura, manifiesto e importaciones
03	Modelos, ORM y campos
04	Vistas, acciones y menús
05	Relaciones entre modelos
06	Lógica Python: campos calculados, onchange, restricciones y botones
07	Seguridad, datos y actualización del módulo
08	Herencia: extender Contactos, Ventas y CRM
09	Flujo de trabajo con Docker y depuración
10	Proyecto continuo PokeStore y criterios de calidad

Objetivo de la UD2
Al terminar debes ser capaz de crear un addon instalable, añadir modelos y vistas propios, programar lógica de negocio en Python y extender módulos estándar sin modificar el código original de Odoo.

1. De usuario a desarrollador de Odoo

Odoo está organizado en módulos. Un módulo puede crear funcionalidad nueva o extender funcionalidad existente. La idea clave es que no editamos el código de Contactos, Ventas o CRM: creamos nuestro propio módulo y declaramos de qué módulos depende.

Necesidad en PokeStore	Estrategia
Gestionar un catálogo propio de figuras	Crear un modelo nuevo: `pokestore.figure`.
Guardar colecciones y relacionarlas con figuras	Crear otro modelo y relacionarlo mediante Many2one/One2many.
Añadir datos de coleccionista a los contactos	Extender `res.partner` con `_inherit`.
Añadir instrucciones de empaquetado en presupuestos	Extender `sale.order`.
Registrar intereses Pokémon en oportunidades comerciales	Extender `crm.lead`.

Regla de arquitectura
Si una funcionalidad pertenece a tu negocio, debe vivir en tu addon. No cambies archivos dentro del contenedor oficial de Odoo ni dentro de los módulos estándar.

2. Dónde vive un módulo

La instalación local de la UD1 monta la carpeta addons que está junto a `compose.yaml` dentro del contenedor como `/mnt/extra-addons`. Por tanto, cada módulo que programes será una carpeta dentro de esa carpeta local.

```
00_Instalacion_local/
├── compose.yaml
└── addons/
    └── pokestore/
        ├── __init__.py
        ├── __manifest__.py
        ├── models/
        ├── security/
        ├── views/
        ├── data/
        └── demo/
```

Odoo reconoce el addon por su carpeta y por el archivo `__manifest__.py`. Los archivos Python forman paquetes, por eso utilizamos `__init__.py` para importar los módulos de código.

> Nombres seguros
> Utiliza minúsculas, números y guiones bajos para la carpeta del módulo: `pokestore`, `aulatec_extension`, etc. Evita espacios, guiones y tildes.

3. El manifiesto y las importaciones

El manifiesto describe el módulo: nombre, versión, dependencias y archivos que Odoo debe cargar. El orden de la lista `data` importa porque los archivos se procesan secuencialmente.

```
{
    "name": "PokeStore",
    "version": "18.0.1.0.0",
    "license": "LGPL-3",
    "depends": ["base"],
    "data": [
        "security/ir.model.access.csv",
        "views/pokestore_figure_views.xml",
        "views/pokestore_menus.xml",
    ],
    "application": True,
    "installable": True,
}
```

En el `__init__.py` de la raíz importamos la carpeta `models`. Dentro de `models/__init__.py` importamos cada archivo Python que contenga modelos.

```
# pokestore/__init__.py
from . import models

# pokestore/models/__init__.py
from . import figure
```

4. Modelos y ORM

Un modelo de Odoo es una clase Python que hereda de `models.Model`. El ORM transforma los campos declarados en Python en columnas y relaciones de PostgreSQL, aplica permisos y permite consultar y modificar registros sin escribir SQL manualmente.

```
from odoo import fields, models

class PokeStoreFigure(models.Model):
    _name = "pokestore.figure"
    _description = "Figura coleccionable"
    _order = "name"

    name = fields.Char(string="Nombre", required=True)
    pokemon = fields.Char(string="Pokémon", required=True)
    price = fields.Float(string="Precio", required=True)
    stock = fields.Integer(string="Stock", default=0)
    active = fields.Boolean(default=True)
```

Elemento	Función
`_name`	Nombre técnico único del nuevo modelo.
`_description`	Descripción legible del modelo.
`_order`	Orden por defecto de los registros.
`fields.*`	Define el tipo, etiqueta y reglas del dato.
Campos automáticos	Odoo añade `id`, fechas y usuarios de creación/modificación.

4.1 Campos habituales

Tipo	Uso PokeStore	Ejemplo
Char	Nombre, referencia o Pokémon	`fields.Char()`
Text	Descripción extensa	`fields.Text()`
Integer	Unidades de stock	`fields.Integer()`
Float	Precio o porcentaje	`fields.Float()`
Boolean	Activa, exclusiva, reservada	`fields.Boolean()`
Date / Datetime	Fecha de lanzamiento/reserva	`fields.Date()`
Selection	Rareza o estado	`fields.Selection([...])`

```
rarity = fields.Selection(
    [
        ("normal", "Normal"),
        ("rare", "Rara"),
        ("exclusive", "Exclusiva"),
    ],
    string="Rareza",
    default="normal",
    required=True,
)
```

> Python sigue siendo Python
> Los modelos contienen lógica Python normal, pero dentro de métodos de Odoo trabajamos con recordsets y con el ORM. No uses `print()` como parte de la lógica de negocio.

5. Seguridad mínima antes de usar un modelo

Crear una clase no concede permiso a los usuarios para abrirla. Los derechos de acceso se declaran normalmente en `security/ir.model.access.csv`.

```
id,name,model_id/id,group_id/id,perm_read,perm_write,perm_create,perm_unlink
access_pokestore_figure_user,pokestore.figure.user,model_pokestore_figure,base.group_user,1,1,1,1
```

El identificador automático del modelo `pokestore.figure` se transforma en `model_pokestore_figure`. En proyectos más avanzados crearemos grupos propios y podremos limitar los permisos de borrado o escritura.

> No confundas menú y permiso
> Ocultar un menú no protege un modelo. La seguridad real debe estar declarada con derechos de acceso y, cuando sea necesario, reglas de registro.

6. Vistas, acciones y menús

La interfaz de Odoo se declara principalmente en XML. En Odoo 18 la vista tabular utiliza el elemento raíz `<list>`. Una acción abre los registros de un modelo y un menú dispara esa acción.

6.1 Vista de lista

```
<record id="pokestore_figure_view_list" model="ir.ui.view">
    <field name="name">pokestore.figure.list</field>
    <field name="model">pokestore.figure</field>
    <field name="arch" type="xml">
        <list string="Figuras">
            <field name="name"/>
            <field name="pokemon"/>
            <field name="rarity"/>
            <field name="price"/>
            <field name="stock"/>
        </list>
    </field>
</record>
```

6.2 Vista de formulario

```
<form string="Figura">
    <sheet>
        <group>
            <group>
                <field name="name"/>
                <field name="pokemon"/>
            </group>
            <group>
                <field name="rarity"/>
                <field name="price"/>
                <field name="stock"/>
            </group>
        </group>
    </sheet>
</form>
```

6.3 Acción y menú

```
<record id="pokestore_figure_action" model="ir.actions.act_window">
    <field name="name">Figuras</field>
    <field name="res_model">pokestore.figure</field>
    <field name="view_mode">list,form</field>
</record>

<menuitem id="pokestore_menu_root" name="PokeStore"/>
<menuitem id="pokestore_menu_catalog" name="Catálogo" parent="pokestore_menu_root"/>
<menuitem id="pokestore_menu_figures" action="pokestore_figure_action" parent="pokestore_menu_catalog"/>
```

7. Relaciones entre modelos

Las relaciones son fundamentales en Odoo. No copies nombres como texto si el dato ya existe como registro: enlázalo.

Relación	Significado	Ejemplo PokeStore
Many2one	Muchos registros apuntan a uno	Muchas figuras pertenecen a una colección.
One2many	Vista inversa de un Many2one	Una colección muestra todas sus figuras.
Many2many	Muchos con muchos	Una figura puede tener varias etiquetas y una etiqueta varias figuras.

```
# En pokestore.figure
collection_id = fields.Many2one(
    "pokestore.collection",
    string="Colección",
    ondelete="restrict",
)

# En pokestore.collection
figure_ids = fields.One2many(
    "pokestore.figure",
    "collection_id",
    string="Figuras",
)
```

Punto clave
El `One2many` no crea una columna adicional: utiliza el campo `Many2one` del modelo relacionado como relación inversa.

8. Lógica de negocio en Python

8.1 Campos calculados

Un campo calculado se obtiene a partir de otros campos. Declaramos sus dependencias para que Odoo sepa cuándo recalcularlo.

```
from odoo import api, fields, models

stock_value = fields.Float(
    string="Valor en stock",
    compute="_compute_stock_value",
    store=True,
)

@api.depends("price", "stock")
def _compute_stock_value(self):
    for figure in self:
        figure.stock_value = figure.price * figure.stock
```

8.2 Onchange

Un `@api.onchange` reacciona en el formulario antes de guardar. Sirve para ayudar al usuario, no para garantizar reglas críticas porque no se ejecuta en todos los contextos posibles.

```
@api.onchange("rarity")
def _onchange_rarity(self):
    if self.rarity == "exclusive" and not self.notes:
        self.notes = "Revisar embalaje especial."
```

8.3 Restricciones

Para impedir datos inválidos utilizamos restricciones. Si el dato no cumple la regla, lanzamos una excepción de validación.

```
from odoo.exceptions import ValidationError

@api.constrains("price", "stock")
def _check_price_and_stock(self):
    for figure in self:
        if figure.price < 0:
            raise ValidationError("El precio no puede ser negativo.")
        if figure.stock < 0:
            raise ValidationError("El stock no puede ser negativo.")
```

8.4 Botones que llaman a métodos

```
def action_mark_sold_out(self):
    for figure in self:
        figure.stock = 0
```

El método anterior puede enlazarse desde un botón XML con `type="object"`. El botón llama a un método del modelo actual.

9. Datos, demostración y orden de carga

Además de código y vistas, un módulo puede crear registros al instalarse. Los datos permanentes suelen ir en `data/` y los de demostración en `demo/`. Ambos deben declararse en el manifiesto.

```
"data": [
    "security/ir.model.access.csv",
    "data/pokestore_collection_data.xml",
    "views/pokestore_figure_views.xml",
    "views/pokestore_menus.xml",
],
"demo": [
    "demo/pokestore_demo.xml",
],
```

> Orden de dependencia
> No puedes referenciar en un XML un registro que todavía no se ha cargado. Coloca primero seguridad y datos de los que dependan las vistas o registros posteriores.

10. Crear frente a extender

Situación	Atributos del modelo
Crear un modelo nuevo	Usas `_name = "pokestore.figure"`.
Extender un modelo existente	Usas `_inherit = "res.partner"` y no defines un nuevo `_name`.
Modelo nuevo que hereda comportamiento	Es un caso distinto y se estudia más adelante; en esta unidad priorizamos la extensión directa.

```
from odoo import fields, models

class ResPartner(models.Model):
    _inherit = "res.partner"

    collector_code = fields.Char(string="Código de coleccionista")
    favorite_pokemon = fields.Char(string="Pokémon favorito")
```

Este código no crea una tabla de contactos paralela. Amplía el modelo estándar `res.partner` con campos adicionales aportados por nuestro módulo.

11. Herencia de vistas

Para mostrar los nuevos campos no copiamos la vista completa de Odoo. Creamos una vista hija, indicamos el `inherit_id` y localizamos el punto donde insertar contenido mediante XPath o mediante

un elemento con `position`.

```
<record id="pokestore_partner_form_inherit" model="ir.ui.view">
    <field name="name">res.partner.form.pokestore</field>
    <field name="model">res.partner</field>
    <field name="inherit_id" ref="base.view_partner_form"/>
    <field name="arch" type="xml">
        <xpath expr="//sheet/group" position="after">
            <group string="PokeStore">
                <field name="collector_code"/>
                <field name="favorite_pokemon"/>
            </group>
        </xpath>
    </field>
</record>
```

> Por qué heredamos la vista
> Si Odoo cambia su vista base, nuestra extensión sigue siendo independiente. Copiar y reemplazar una vista estándar hace el mantenimiento mucho más frágil.

Activa el modo desarrollador para inspeccionar una vista y localizar su identificador externo y su estructura antes de escribir el XPath. No supongas que todas las vistas estándar tienen la misma estructura.

12. Extender Contactos, Ventas y CRM

12.1 Contactos

El modelo técnico de Contactos es `res.partner`. PokeStore puede añadir el nivel de coleccionista, Pokémon favorito o preferencias de contacto. Como pertenece al módulo base, `base` ya cubre esta dependencia.

12.2 Ventas

Para personalizar presupuestos y pedidos se extiende `sale.order`. El módulo debe declarar la dependencia correspondiente antes de que Odoo cargue nuestro código.

```
"depends": ["base", "sale"],
```

Ejemplos útiles: indicar si el pedido requiere embalaje de coleccionista, evento de entrega, número de caja protectora o notas específicas de la colección.

12.3 CRM

Las oportunidades comerciales viven en `crm.lead`. Nuestro módulo puede añadir colección de interés, Pokémon buscado o número aproximado de figuras que el cliente quiere reservar.

```
"depends": ["base", "sale", "crm"],
```

> Dependencia = contrato
> Si tu módulo importa o hereda un modelo de otro addon, ese addon debe estar en `depends`. Así Odoo garantiza el orden de carga.

13. Flujo de trabajo en el entorno Docker de la UD1

Ciclo recomendado de desarrollo

1. Crea o modifica los archivos del módulo dentro de la carpeta local `addons`.
2. Si has cambiado Python, reinicia el servicio con `docker compose restart web`.
3. En Odoo, activa el modo desarrollador cuando necesites herramientas técnicas.
4. En Aplicaciones, actualiza la lista de aplicaciones cuando el módulo sea nuevo.
5. Instala el módulo la primera vez; después utiliza Actualizar/Upgrade cuando cambien modelos, vistas, seguridad o datos.
6. Si aparece un error, consulta primero el log con `docker compose logs -f web`.

```
docker compose ps
docker compose restart web
docker compose logs -f web
```

Puerto 8069 o 8070

El puerto del navegador no afecta al desarrollo del addon. Usa la dirección que configuraste en la UD1: `http://localhost:8069` o `http://localhost:8070`.

14. Errores que debes aprender a leer

Mensaje o síntoma	Qué revisar primero
El módulo no aparece	Carpeta dentro de addons, manifiesto válido, lista de apps actualizada y filtro de Apps.
ImportError / ModuleNotFoundError	Cadena de `__init__.py` e imports.
Model not found / KeyError	Nombre técnico, dependencia del módulo y reinicio tras cambios Python.
External ID not found	Orden en manifest, dependencia o XML ID incorrecto.
Element cannot be located in parent view	XPath o vista padre incorrectos; inspecciona la vista real en modo desarrollador.
AccessError	ACL o grupos insuficientes.
ParseError	XML mal formado o registro inválido.

No corrijas varios archivos a la vez sin volver a probar. El log suele indicar el archivo y la línea aproximada donde Odoo dejó de cargar el módulo.

15. Proyecto continuo PokeStore

Los tutoriales de esta unidad hacen crecer un único módulo. La meta no es memorizar una plantilla, sino entender qué capa estás modificando en cada paso.

Tutorial	Evolución del módulo
T01	Módulo instalable con el modelo Figura, seguridad, lista, formulario, acción y menús.
T02	Colecciones, etiquetas, relaciones y mejores vistas.
T03	Campos calculados, onchange, restricciones y botones Python.
T04	Grupos, permisos, datos maestros y datos demo.
T05	Extensión de Contactos con perfil de coleccionista.
T06	Extensión de Ventas y CRM e integración entre módulos.

Tutorial	Evolución del módulo
T07	Reto guiado: convertir PokeStore en un addon coherente y mantenible.

Trabajo incremental
Haz una copia o un commit cuando un paso funcione. Si una ampliación rompe el módulo, debes poder volver al último punto estable.

16. Criterios de calidad

- Nombres técnicos en `snake_case` y nombres de modelos con puntos: `pokestore.figure`.
- Un archivo por modelo cuando el proyecto empieza a crecer.
- Sin SQL directo para operaciones normales: utiliza el ORM.
- Sin `print()` dentro de los métodos de negocio.
- Las validaciones importantes se implementan como restricciones, no solo con cambios visuales del formulario.
- No dupliques registros estándar: relaciónalos mediante campos relacionales.
- No modifiques módulos oficiales: extiéndelos desde tu addon.
- Todo archivo XML/CSV que deba cargarse debe aparecer en el manifiesto.
- Las dependencias deben ser explícitas.
- Tras cada cambio, comprueba instalación/actualización y revisa el log si falla.

Referencias técnicas

Esta unidad toma como referencia Odoo 18 Community y la documentación oficial de desarrollo: `https://www.odoo.com/documentation/18.0/developer/tutorials/server_framework_101.html` y `https://www.odoo.com/documentation/18.0/developer/reference/backend/orm.html`.

Siguiente paso
Empieza por el Tutorial 01. No avances al Tutorial 02 hasta que PokeStore aparezca en Odoo, puedas crear figuras y el módulo se pueda actualizar sin errores.